

Introduction

In this documentation we will provide instructions on how to access RGTFun, an example calculation using RGTFun that can be used as a tutorial for new users, and then discuss the included testing suite. Documentation for all functions can be found in the appendix.

Downloading RGTFun

RGTFun can be downloaded from the public Github repository linked here:

<https://github.com/nbb2/rgtfun/tree/paper>

Please download all folders from the repository and ensure that they are all located within a *RGTFun* folder on your machine (it does not have to be called *RGTFun*). This is important because the app will ask you to select the RGTFun folder on your machine so it can establish the path to the *src* and *gui* folders. It does not matter where your *RGTFun* folder is located as long as it is a local folder, i.e. not in a cloud service. Once downloading the repository folders, you can start the app by opening the *gui.mlapp* file in the *gui* folder. Note that RGTFun must be run with MATLAB R2023b or newer. RGTFun is also available as a standalone desktop app and can be downloaded from the Releases page in the Github repository. Note that the standalone desktop app requires MATLAB Runtime R2024a to be installed, which is free to download from MathWorks (install instructions are located on the Release page in the RGTFun repository).

Downloading Chebfun

Chebfun must be downloaded locally in order to run RGTFun. Chebfun can be downloaded from the Chebfun website linked here:

<https://www.chebfun.org/download/>

Note that RGTFun requires Chebfun V5.7.0 or newer. Please download the Chebfun .zip file and unzip on your machine. We recommend putting Chebfun in an easy-to-find location since RGTFun will ask you to provide the path to the Chebfun directory.

An Example Calculation of Argon-Argon Interaction

We will now present example calculations of transport quantities and scattering integrals using RGTFun. The calculations will be performed for an Argon-Argon ZBL potential.

Timing (end-to-end tutorial)

On a typical laptop, this end-to-end tutorial takes **~5–10 minutes** to complete (including generating inputs, running each tab, and saving plots/files).

Timing reference: Apple Silicon laptop (macOS), MATLAB R2025b.

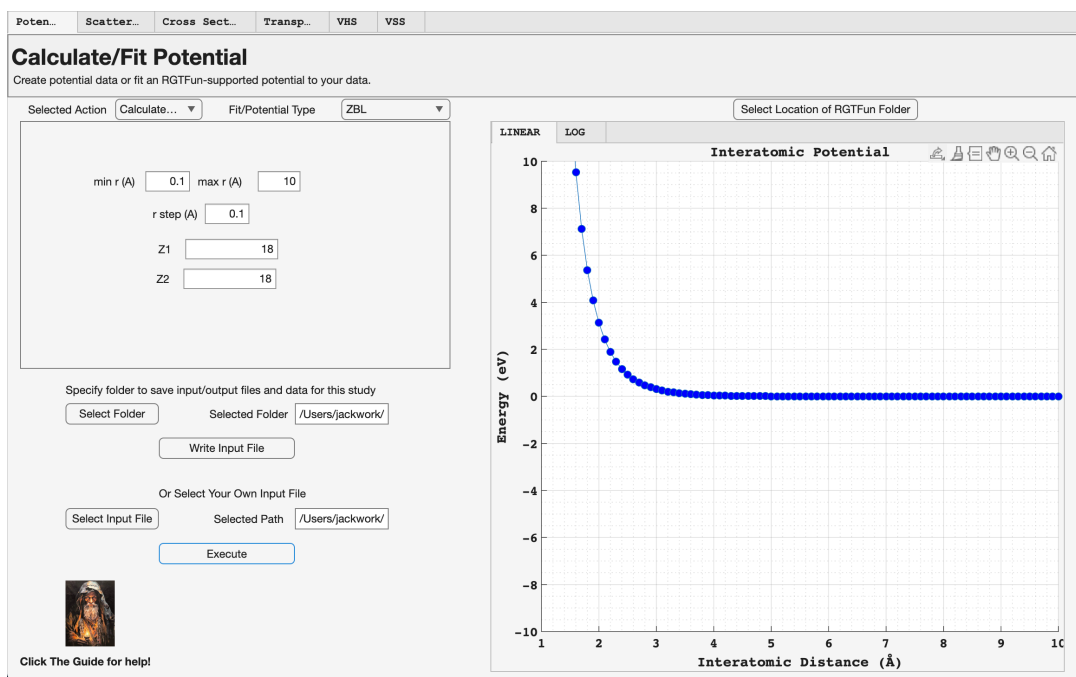
What affects runtime most

- The numerical grids used in the scattering and cross-section steps:
 - runtime increases roughly with **(# energy points) × (# impact parameter points)** in the scattering-integrals step.
- The number of temperature points used in the transport-coefficient step.

Speed-up for quick iteration

If you want a faster “sanity check” run, reduce the grid sizes (e.g., fewer energy points or fewer impact-parameter points) and then increase them for final-quality results.

Calculate/Fit Potential Tab



Screenshot of *Calculate/Fit Potential* tab for an Argon-Argon ZBL potential.

The *Calculate/Fit Potential* tab allows you to either create your own potential data or fit one of the RGTFun-supported potentials to your own data.

Inputs used in this tutorial

Setting / Parameter	Value	Notes
Selected Action	Calculate Potential	Creates user-specified potential data
Fit/Potential Type	ZBL	Universal screened potential
min r (Å)	0.1	Minimum interatomic distance
max r (Å)	10	Maximum interatomic distance
r step (Å)	0.1	Distance grid spacing
Z1	18	Argon atomic number
Z2	18	Argon atomic number

Steps

1. Open the **Calculate/Fit Potential** tab.
2. In **Selected Action**, choose **Calculate Potential**.
3. In **Fit/Potential Type**, choose **ZBL**.
4. Enter the values in the table above: **min r**, **max r**, **r step**, **Z1**, and **Z2**.
5. Under *Specify folder to save input/output files and data for this study*, click **Select Folder** and choose an output directory.
6. Click **Write Input File** to generate an input file with the parameters above.
 - This will automatically populate the *Or Select Your Own Input File* field.
7. Click **Execute** to:
 - generate the potential data,
 - plot the potential,
 - save the plot image, and
 - write the output data and fit file to the selected directory.

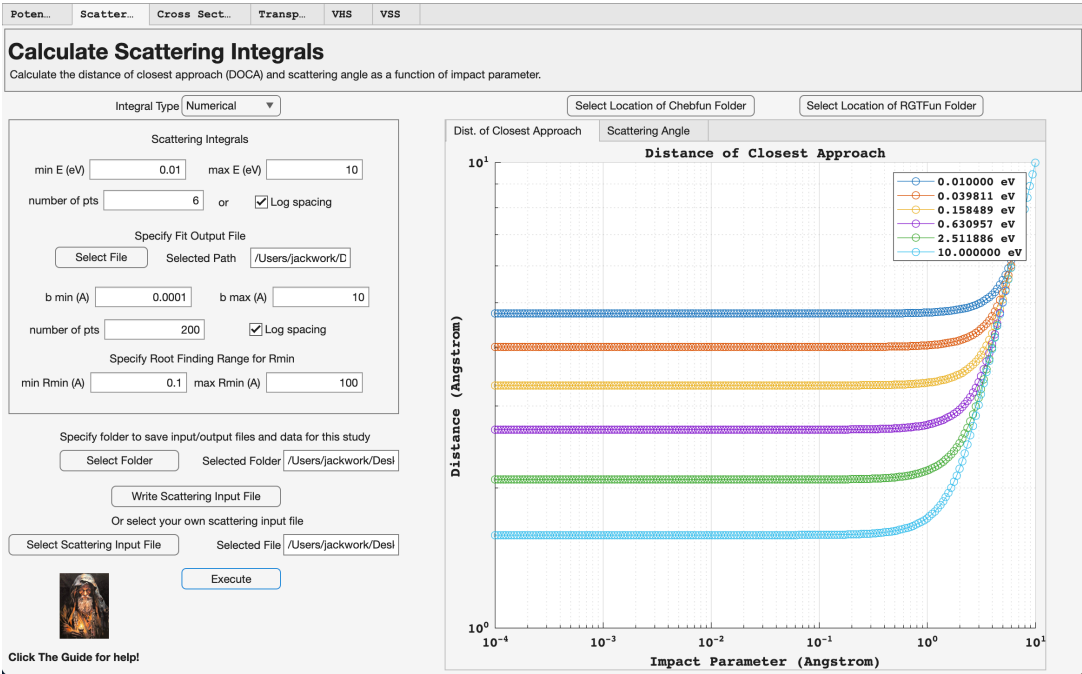
Expected output

The resulting plot should match the linear plot in the figure above.

Notes

- Distances are specified in **Å**.
- For other species, set **Z1** and **Z2** to the corresponding atomic numbers.

Calculate Scattering Integrals Tab



Screenshot of *Calculate Scattering Integrals* tab for an Argon-Argon ZBL potential.

The *Calculate Scattering Integrals* tab allows you to calculate the distance of closest approach (DOCA) and scattering angle as a function of impact parameter.

Inputs used in this tutorial

Setting / Parameter	Value	Notes
Integral Type	Numerical	Uses potential data generated in the previous tab
min E	0.01	Energy range lower bound
max E	10	Energy range upper bound
Energy spacing	Log	Check Log spacing
number of pts (E)	50	Number of energies evaluated
Fit output file	<i>(auto-populated)</i>	Populated after clicking Execute in the previous tab
b min	0.0001	Impact parameter range lower bound
b max	10	Impact parameter range upper bound
b spacing	Log	Use log spacing
number of pts (b)	200	Number of impact parameters evaluated
min Rmin	0.1	Root-solver bracket for DOCA
max Rmin	100	Root-solver bracket for DOCA

Steps

1. Open the **Calculate Scattering Integrals** tab.
2. Set **Integral Type** to **Numerical** (to use the potential data generated in the previous tab).
3. Under the energy settings, enter:
 - **min E = 0.01**
 - **max E = 10**
 - check **Log spacing**
 - set **number of pts = 6**
4. Confirm that the **fit output file** field is populated (this should happen automatically after running **Execute** in the previous tab). If it is not populated, select the fit output file manually.
5. Under the impact parameter settings, enter:
 - **b min = 0.0001**
 - **b max = 10**
 - select **log spacing**
 - set **number of points = 200**
6. Under the DOCA root-solver settings, set:
 - **min Rmin = 0.1**
 - **max Rmin = 100**
7. Click **Write Scattering Input File** to generate the input file for this tab.
8. Click **Execute** to:
 - compute DOCA and scattering angle datasets,
 - save the datasets to the selected study/output folder, and
 - save the generated figures as image files.

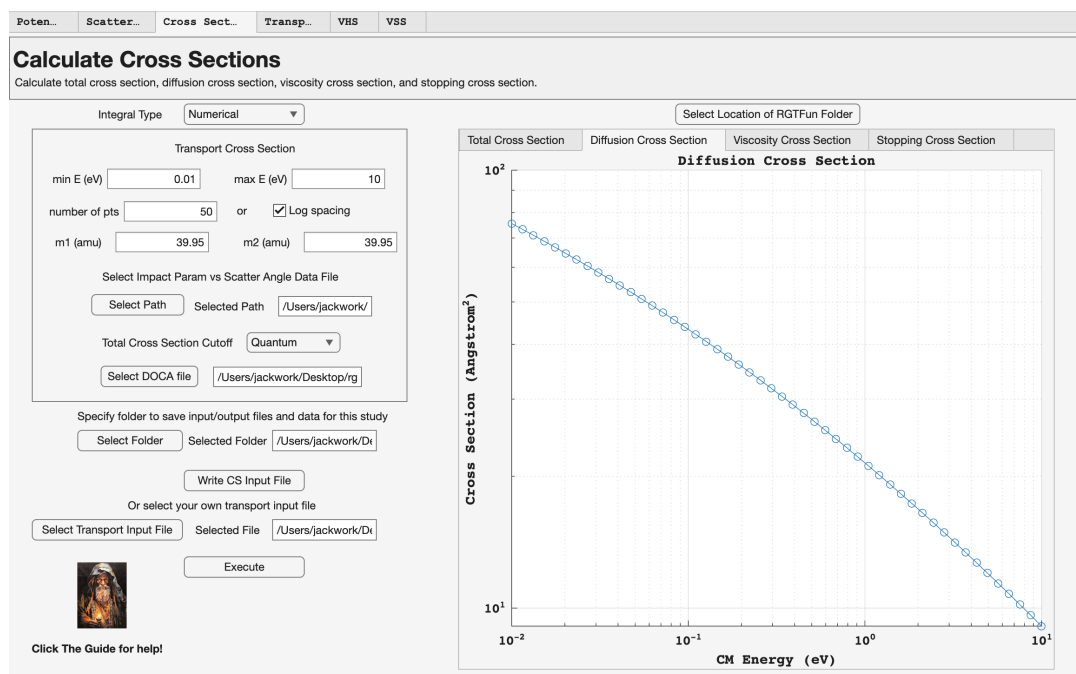
Expected output

After clicking **Execute**, you should see plots for the DOCA and scattering angle. The DOCA plot should have a similar overall shape to the figure above. Note that the figure above was generated using **6 energy points** to make individual curves easier to distinguish; for this tutorial we recommend using **50 energy points** to obtain a smoother, better-resolved energy dependence. The tab will also write the DOCA and scattering-angle data files into your study folder.

Notes

- If the DOCA solver fails or produces nonphysical values, adjust **min Rmin** and **max Rmin** to better bracket the turning point for your potential and energy range.

Calculate Cross Sections Tab



Screenshot of *Calculate Cross Sections* tab for an Argon-Argon ZBL potential.

The *Calculate Cross Sections* tab allows you to calculate total cross section, diffusion cross section, viscosity cross section, and stopping cross section.

Inputs used in this tutorial

Setting / Parameter	Value	Notes
Integral Type	Numerical	Uses scattering angle vs. impact parameter from the previous tab
Energy grid (min E, max E, number of pts, spacing)	(auto-populated)	Must match the scattering integrals tab parameters
m1 (amu)	39.95	Argon atomic mass
m2 (amu)	39.95	Argon atomic mass
Scattering-angle data file	(auto-populated)	Populated after running the previous tab
Total cross section cutoff	Quantum mechanical cutoff	Used for the total cross section

Steps

1. Open the **Calculate Cross Sections** tab.
2. Set **Integral Type** to **Numerical** to use the scattering-angle data from the previous tab.
3. Confirm the energy grid fields are auto-populated (min E, max E, number of points, and spacing).
 - **Do not change these values** for this tutorial. They must match the energy grid used to generate the scattering angle data.
4. Confirm the **scattering angle data file path** is populated. If it is not, select the file manually.
5. Enter the species masses:
 - **m1 = 39.95 amu**
 - **m2 = 39.95 amu**
6. For the **total cross section**, select the **quantum mechanical cutoff** option.
7. Click **Write CS Input File** to generate the cross-section input file.
8. Click **Execute** to:
 - compute the requested cross sections,
 - save the outputs as CSV files in your study/output folder, and
 - save the generated figures as image files.

Expected output

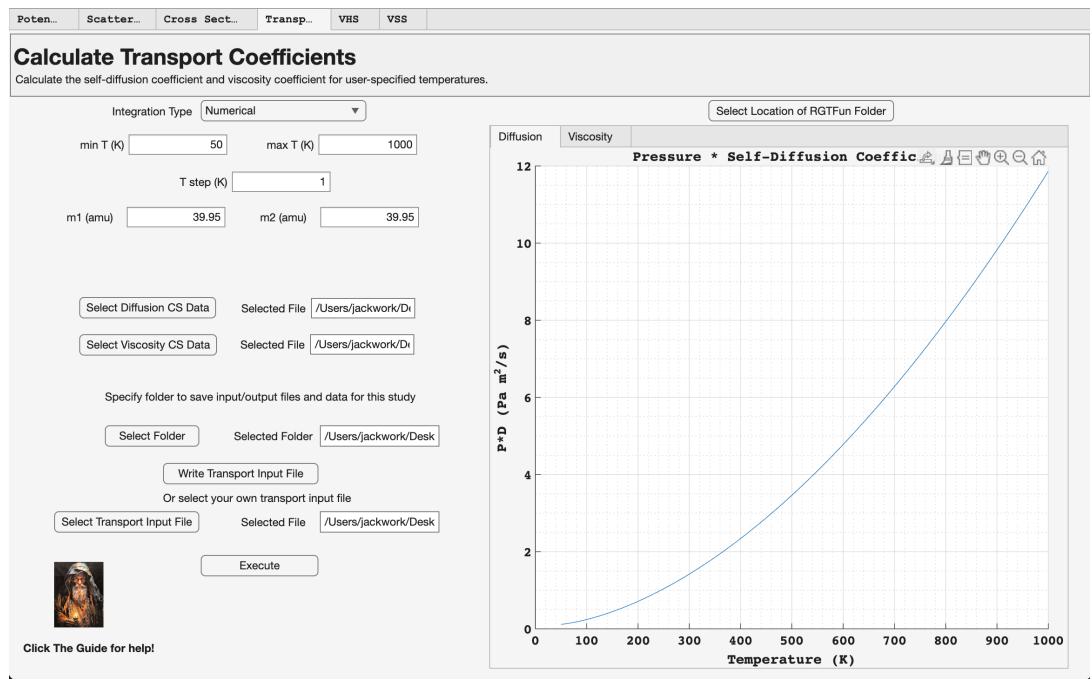
After clicking **Execute**, RGTFun will generate plots and CSV files for the selected cross sections (total, diffusion, viscosity, and/or stopping) as functions of center-of-mass energy. The diffusion cross section is the quantity shown in the figure above and should match your result.

Notes

- The **energy grid must match** the scattering angle dataset used as input. If you change the energy grid here without regenerating the scattering angle data, the results will be inconsistent and the code will fail.

- If you change the masses (m_1/m_2) to another species pair, ensure they are in **atomic mass units (amu)**.

Calculate Transport Coefficients Tab



Screenshot of *Calculate Transport Coefficients* tab for an Argon-Argon ZBL potential.

The *Calculate Transport Coefficients* tab allows you to calculate the self-diffusion coefficient and viscosity coefficient for user-specified temperatures.

Inputs used in this tutorial

Setting / Parameter	Value	Notes
Integration Type	Numerical	Uses cross section data from the previous tab
min T (K)	50	Lower temperature bound
max T (K)	1000	Upper temperature bound
T step (K)	1	Temperature increment
Species masses	(auto-populated)	Populated from the previous tab
Cross section file paths	(auto-populated)	Populated from the previous tab

Steps

1. Open the **Calculate Transport Coefficients** tab.
2. Set **Integration Type** to **Numerical** to use cross section data generated in the previous tab.
3. Enter the temperature grid (Kelvin):
 - **min T = 50**
 - **max T = 1000**
 - **T step = 1**
4. Confirm that the **species masses** and **cross section file paths** are populated (these should auto-populate from the previous tab). If they are not populated, return to the Cross Sections tab and re-run **Execute**, or select the files manually.
5. Click **Write Transport Input File** to write the input file into your study/output folder.
6. Click **Execute** to:
 - compute self-diffusion and viscosity coefficients over the temperature grid,
 - save the results as CSV files in your study/output folder, and
 - generate and save the corresponding plots as image files.

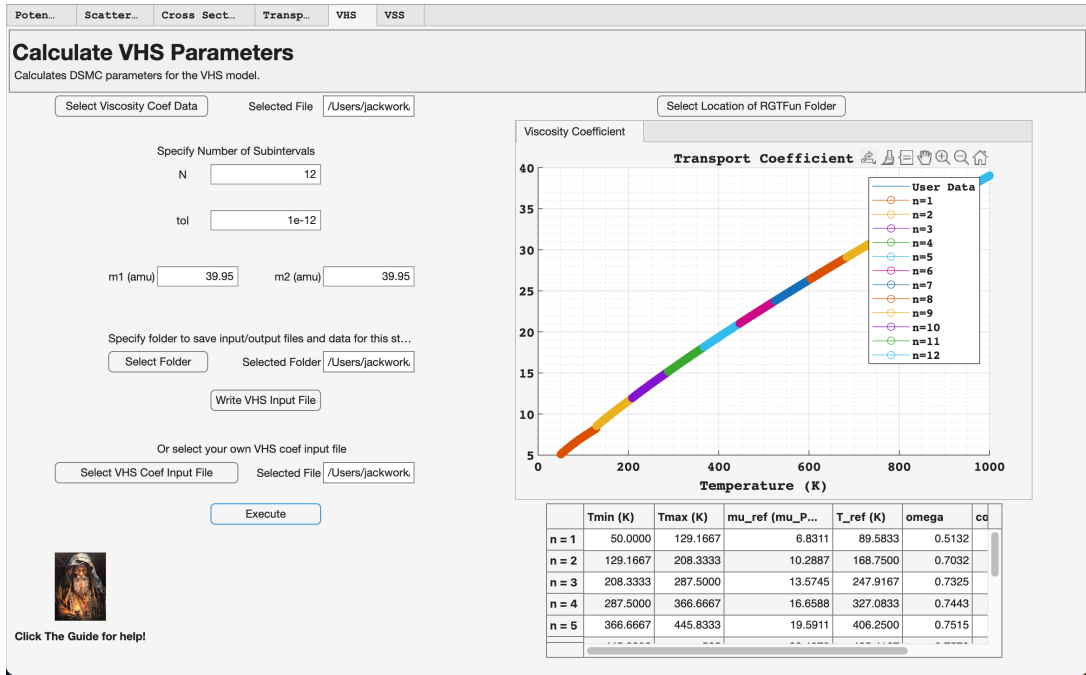
Expected output

After clicking **Execute**, RGTFun will produce temperature-dependent plots for the self-diffusion and viscosity coefficients and write the corresponding CSV output files to the study folder. The plot for the self-diffusion coefficient is shown in the figure above and should match your result.

Notes

- Temperatures are specified in **Kelvin**.

Calculate VHS Parameters Tab



Screenshot of *Calculate VHS Parameters* tab for an Argon-Argon ZBL potential.

The *Calculate VHS Parameters* tab allows the user to calculate the ω parameter for the VHS DSMC model. This parameter is calculated by fitting the VHS diffusion coefficient expression to the user-provided viscosity coefficient data.

Inputs used in this tutorial

Setting / Parameter	Value	Notes
Viscosity coefficient file	<i>(auto-populated)</i>	Populated after executing the Transport Coefficients tab
N (subintervals)	12	Viscosity data are split into N subintervals; a fit is performed per subinterval
Fit tolerance (tol)	1e-12	Suggested: 1e-12 or smaller
Species masses	<i>(auto-populated)</i>	Populated from previous tab

Steps

1. Open the **Calculate VHS Parameters** tab.
2. Confirm that the **viscosity coefficient data file** path is populated (this should auto-populate after running the previous tab). If it is not populated, select the viscosity CSV file manually.
3. Set the number of fitting subintervals:
 - **N = 12**
4. Set the fitting tolerance:
 - **tol = 1e-12**
5. Confirm that the **species masses** are populated (they should carry over from the previous tab).
6. Click **Write VHS Input File** to write the input file into your study/output folder.
7. Click **Execute** to:
 - fit ω over each subinterval of the viscosity data,
 - compute a collision diameter for each subinterval,
 - plot the viscosity data and fitted subintervals, and
 - populate the tab's results table.

Expected output

After clicking **Execute**, the tab will display a plot showing the input viscosity data and the fitted subintervals, and the results table will list ω and collision diameter values for each subinterval. The results table is also saved to the study/output folder. The figure above shows a plot of the fitted viscosity coefficient data and the fitted ω values.

Notes

- Increasing **N** increases the resolution of the fit (more subintervals), but may amplify noise in the fitted parameters if the viscosity data are not smooth.

- If the fit fails to converge, try relaxing the tolerance slightly (e.g., 1e-10) or reducing **N**.

Calculate VSS Parameters Tab

Poten...

Scatter...

Cross Sect...

Transp...

VSS

VSS

Calculate VSS Parameters

Calculates DSMC parameters for the VSS model.

Select Diffusion Coef Data

Select VHS Coef Table

Selected File

Selected File

/Users/jackwork/C

/Users/jackwork/L

Select Location of RGTFun Folder

alpha guess

1.3

least squares tol

1e-12

m1 (amu)

39.95

m2 (amu)

39.95

Specify folder to save input/output files and data for this study

Select Folder

Selected Folder

/Users/jackwork/De

Write VSS Input File

Or select your own VSS coef input file

Select VSS Coef Input File

Selected File

/Users/jackwork/De

Execute



Click The Guide for help!

Reference Temp (K)	alpha	collision_diam (A)
89.5833	1.6697	4.8080
168.7500	1.5924	4.9794
247.9167	1.5693	4.8354
327.0833	1.5676	4.7036
406.2500	1.5721	4.5948
485.4167	1.5785	4.5039
564.5833	1.5853	4.4263
643.7500	1.5920	4.3589
722.9167	1.5984	4.2992
802.0833	1.6044	4.2459
881.2500	1.6100	4.1976
960.4167	1.6154	4.1535

Screenshot of *Calculate VSS Parameters* tab for an Argon-Argon ZBL potential.

The *Calculate VSS Parameters* tab computes Variable Soft Sphere (VSS) DSMC parameters from previously generated transport data. Using (i) the reference viscosity and diffusion coefficients and (ii) the VHS fit results (reference ω values), this tab solves for the VSS parameters α and the **collision diameter** as functions of the reference temperature.

Given an initial guess for α , RGTFun iterates between:

- computing the collision diameter from the viscosity data and current α and ω , and
- updating α from the diffusion data and the updated collision diameter,

until the user-specified least-squares tolerance is satisfied (or a maximum iteration count is reached). The outputs are saved to the study folder and include the reference temperature, fitted α , and collision diameter.

Inputs used in this tutorial

Setting / Parameter	Value	Notes
Diffusion coefficient file	(auto-populated)	From the Transport Coefficients tab
VHS coefficient table	(auto-populated)	From the VHS Parameters tab
alpha guess	1.3	Recommended range: 1–2
least squares tol	1e-12	Suggested: 1e-12 or smaller
Species masses	(auto-populated)	Carried over from previous tabs

Steps

1. Open the **Calculate VSS Parameters** tab.
2. Confirm that the **diffusion coefficient data file** path and **VHS coefficient table** path are populated.
 - If not, select the diffusion CSV (from the Transport tab) and the VHS table CSV (from the VHS tab) manually.
3. Enter an initial guess for α (recommended range 1–2):
 - **alpha guess = 1.3**
4. Set the least squares tolerance:
 - **tol = 1e-12**
5. Confirm that the **species masses** are populated.
6. Click **Write VSS Input File** to write the input file into your study/output folder.
7. Click **Execute** to:
 - compute α and collision diameter values at each reference temperature, and

- populate the tab's results table.

Expected output

After clicking **Execute**, the results table will list α and collision diameter values for each reference temperature. The table is also written to the study folder as. The figure above shows the calculated α and collision diameter values.

Notes

- If the iteration fails to converge, try adjusting **alpha guess** (e.g., 1.1–1.8) and/or relaxing **tol** (e.g., 1e-10).
- The VSS fit assumes the diffusion and viscosity data correspond to the same species pair and reference temperatures used in the earlier tabs.

RGTFun Test Suite

A test suite has been included in the main RGTFun distribution so that users can verify their version is functioning correctly. Test functions were written to verify the functionality of all main functions within RGTFun. The test functions are located in the *test* folder within the main *RGTFun* folder. All test reference data is located in the *testFiles* folder. To run the tests, please change directories to the *test* folder and load the test functions in the MATLAB Test Browser. Then run the current suite, and verify that all tests were executed successfully.

Community Guidelines

Please put all requests for code changes in the Github issue tracker at:
<https://github.com/nbb2/rgtfun/issues>

Appendix:Function Documentation

Table of Contents

- [DOCAroot](#)
- [GMquadScatteringAngle](#)
- [VHScoef](#)
- [VHSviscosity](#)
- [VSSalpha](#)
- [VSScoef](#)
- [VSSconvergence](#)
- [VSSdiameter](#)
- [VSSdiffusion](#)
- [coulomb](#)
- [coulombchar](#)
- [diffusion_coef](#)
- [diffusioncs](#)
- [difscatter](#)
- [distclose](#)
- [impact](#)
- [lj_124](#)
- [lj_126](#)
- [magicscatter](#)
- [morse](#)
- [numdiffusioncoef](#)
- [numdiffusioncs](#)
- [numstoppingcs](#)
- [numtotalcs](#)
- [numtotalcscheb](#)
- [numvisccoef](#)
- [numvisccs](#)
- [powerlaw](#)
- [run_calcpotential](#)

- [run_dsmccoef](#)
- [run_fitpotential](#)
- [run_scatteringintegrals](#)
- [run_transportCS](#)
- [run_transportcoefs](#)
- [run_vhscoef](#)
- [run_vsscoef](#)
- [visc_coef](#)
- [viscositycs](#)
- [zbl](#)
- [zblchar](#)
- [zblderivative](#)

DOCARoot

```
function r0 = DOCARoot(Ec,b,V,rmin,rmax,chebfunpath)
%DOCAROOT    Outputs float value for the root of the DOCA equation.
%   Y=DOCAROOT(EC,B,V,RMIN,RMAX) generates a float value for the root
%   of the distance of closest approach (DOCA) equation using two
%   iterations of the CHEBFUN root finding package.
%
%   -- EC must be the the energy in eV.
%   -- B must be the impact paramter in Angstrom.
%   -- V must be the potential function handle.
%   -- RMIN is the lower bound of possible root values in Angstrom.
%   -- RMAX is the upper bound of possble root values in Angstrom.
%
%   See also GMQUADSCATTERINGANGLE RUN_SCATTERINGINTEGRALS
addpath(chebfunpath)
chebDOCA = chebfun(@(r) doca(r,Ec,b,V),[rmin,rmax]);
ri = max(roots(chebDOCA));
chebDOCA2 = chebfun(@(r) doca(r,Ec,b,V),[0.9*ri,1.1*ri]);
r0 = max(roots(chebDOCA2));

end

function d = doca(r,Ec,b,V)
%MY_DOCA Outputs the DOCA equation.
%   Y=MY_DOCA(R,EC,B,V) generates the DOCA equation.
%   -- R is a placeholder value for distance.
%   -- EC must be the energy in eV.
%   -- B must be the impact parameter in Angstrom.
%   -- V is the potential function handle.
%
%   See also DOCAROOT
d = r^2 - ((r^2)*V(r)/Ec) - (b^2);
end
```

```

function th = GMquadScatteringAngle(V,Ec,b,rm,n)
%GMQUADSCATTERINGANGLE    Outputs float scattering angle.
%   Y=GMQUADSCATTERINGANGLE(V,EC,B,RM,N) generates a float value for the
%   scattering angle using GM Quadrature.
%
%   -- V must be the potential function handle.
%   -- EC must be the the energy in eV.
%   -- B must be the impact paramter in Angstrom.
%   -- RM is the distance of closest approach in Angstrom.
%   -- N is the number of trapezoids to use.
%
%   See also DOCAROOT RUN_SCATTERINGINTEGRALS
sum = 0;
for j = 1:(n/2)
    anj = cos(((2*((n/2) - j + 1) - 1)/(2*n))*pi);
    aj = cos(((2*j - 1)/(2*n))*pi);
    gj = ( 1 - (V(rm/aj)/Ec) - ((b^2 * aj^2)/(rm^2)))^(-1/2);
    sum = sum + (anj * gj);
end
th = pi * (1 - ((2*b*sum)/(n*rm)));
end

```

Published with MATLAB® R2024a

VHScoef

```

function y = VHScoef(fitt,T_sample,mu_sample,excludeT,vq,tol)
%VHSCOEF    Outputs sample visc value and VHS param.
%   Y=VHSCOEF(FITT,T_SAMPLE,EXCLUDET,VQ,TOL) outputs a reference
%   viscosity value and VHS parameter omega by fitting the VHS model to
%   user-specified visosity coefficient data.
%
%   -- FITT must be the interpolated temperature values in K.
%   -- T_SAMPLE must be the reference temperature value.
%   -- EXCLUDET must be a logical array of what temp values to use.
%   -- VQ must be the interpolated viscosity coefficient values with the
%   same dimension as FITT.
%   -- TOL must be the fitting tolerance.
%
%   See also RUN_DSMCCOEF
fitcoef = vq;
fitcoef(excludeT) = [];
fitchar = sprintf("(%f)*(T/(%f)).^(a)",mu_sample,T_sample);
ft = fitttype(fitchar,dependent="y",...
    independent="T",coefficients="a");
coeffit = fit(fitt',fitcoef',ft,'TolFun',tol);
omega = coeffvalues(coeffit);
y = [mu_sample,omega];
end

```

Published with MATLAB® R2024a

VHSviscosity

```

function y = VHSviscosity(T,omega,T_ref,mu_ref)
%VHSVISCOSITY Outputs float array with viscosity coefficient values.
% Y=VHSVISCOSITY(T,OMEGA,T_REF,MU_REF) generates a float array
% containing viscosity coefficient value for each value of T using the
% VHS model.
%
% -- T must be a float array of temperature values in K.
% -- OMEGA must be the VHS parameter.
% -- T_REF must be the reference temp in K.
% -- MU_REF must be the reference viscosity value.
%
% See also RUN_DSMCCOEF
y = mu_ref*(T/T_ref).^omega;
end

```

Published with MATLAB® R2024a

VSSalpha

```

function y = VSSalpha(omegavals,m,molar,Tvals,difvals,diams)
%VSSALPHA Outputs VSS param alpha.
% Y=VSSALPHA(OMEGAVALS,M,MOLAR,TVALS,DIFVALS,DIAMS) outputs VSS parameter
% alpha using reference diffusion coefficient and collision diameter data.
%
% -- OMEGAVALS must be an array of reference omega values.
% -- M must be the mass of a particle in kg.
% -- MOLAR must be the molar mass in kg.
% -- TVALS must be an array of the reference temperature values in K.
% -- DIFVALS must be an array of the reference diffusion coefficient data.
% -- DIAMS must be an array of the reference collision diameter in angstrom.
%
% See also RUN_VSSCOEF
kb = 1.380649E-23; %J/K
Na = 6.022E23;
rhoD = molar*difvals./(Na*kb*Tvals);
y = ((4*(5-2*omegavals).*rhoD.*(diams.^2))./(3*sqrt(m*kb*Tvals/pi))) - 1;
end

```

Published with MATLAB® R2024a

VSScoef

```

function y = VSScoef(minT,maxT,Tfine,vq,m,omega,P,tol)
%VSSCOEF Outputs VSS params alpha and d.
% Y=VSSCOEF(MINT,MAXT,TFINE,VQ,M,OMEGA,P,TOL) outputs VSS parameters
% alpha and d by fitting the VSS model to user-specified diffusin
% coefficient data.
%
% -- MINT must be the lower bound of the temp range in K.
% -- MAXT must be the upper bound of the temp range in K.
% -- TFINE must be the interpolated temperature values in K.
% -- VQ must be the interpolated diffusion coefficient values with the
% same dimension as TFINE.

```

```

% -- M must be the reduced mass of the system in amu.
% -- OMEGA must be the VHS param.
% -- TOL must be the fitting tolerance.
%
% See also RUN_DSMCCOEF
kb = 8.617333262E-5; %eV/K
excludeT = ((Tfine < minT) | (Tfine > maxT));
fitcoef = vq;
fitT = Tfine;
fitT(excludeT) = [];
fitcoef(excludeT) = [];
T_sample = 0.5*(minT + maxT);
cref = 2*(2.5-omega)*kb*T_sample/m;
fitchar = sprintf('157377.3718*30000*(a+1)*(pi^0.5)*((%e)*T).*((pi*2*(%e)*T/(%e)).^(%e))/(16*a*gamma(3.5-%e)*(%e)*pi*(d^2)*(%e)');
    kb,kb,m,omega,omega,P,cref,omega);
ft = fitttype(fitchar,dependent="y",...
    independent="T",coefficients=["a" "d"]);
coeffit = fit(fitT',fitcoef',ft,'TolFun',tol,'Lower',[0 0],...
    'Upper',[10 10],'StartPoint',[1 1]);
coefs = coeffvalues(coeffit);
alpha = coefs(1);
d = coefs(2);
y = [alpha,d];
end

```

Published with MATLAB® R2024a

VSSconvergence

```

function y = VSSconvergence(alphavals,omegavals,m,Tvals,difvals,muref,tol)
%VSSCONVERGENCE Checks least squares convergence of VSS parameters.
% Y=VSSCONVERGENCE(ALPHAVALS,OMEGAVALS,M,TVALS,DIFVALS,MUREF,TOL) outputs
% TRUE or FALSE depending on if the user-specified least-squares
% tolerance is satisfied.
%
% -- ALPHAVALS must be an array of the alpha values at each reference temperature.
% -- OMEGAVALS must be an array of the omega values at each reference
% temperature.
% -- M must be the mass of a particle in kg.
% -- TVALS must be an array of the reference temperature values in K.
% -- DIFVALS must be an array of the reference diffusion coefficient data.
% -- MUREF must be an array of the reference viscosity coefficient data.
% -- TOL must be the user-specified least squares tolerance.
% See also RUN_VSSCOEF
kb = 1.380649E-23; %J/K
Na = 6.022E23;
rhoD = m*difvals./(Na*kb*Tvals);
muvals = ((10./alphavals)+5).*rhoD./(21-6*omegavals);
Sc = rhoD./muref; %Schmidt number
LSError = sum((muvals - muref).^2);
y = (LSError > tol);
end

```

Published with MATLAB® R2024a

VSSdiameter

```

function y = VSSdiameter(alphavals,omegavals,m,Tvals,muvals)
%VSSDIAMETER Outputs VSS collision diameter.
% Y=VSSDIAMETER(ALPHAVALS,OMEGAVALS,M,TVALS,MUVALS) outputs VSS collision
% diameter using reference viscosity coefficient and VHS parameter data.
%
% -- ALPHAVALS must be an array of the alpha values at each reference temperature.
% -- OMEGAVALS must be an array of reference omega values.
% -- M must be the mass of a particle in kg.
% -- TVALS must be an array of the reference temperature values in K.
% -- MUVALS must be an array of the reference viscosity coefficient data.
%
% See also RUN_VSSCOEF
kb = 1.380649E-23; %J/K
y2 = ((5*(alphavals+1).*(alphavals+2).*sqrt(m*kb*Tvals/pi))./...
    (4*alphavals.*(5-2*omegavals).*(7-2*omegavals).*muvals)); % meters
y = sqrt(y2);
end

```

Published with MATLAB® R2024a

VSSdiffusion

```

function y = VSSdiffusion(T,omega,a,d,m,P,T_sample)
%MY_VSSDIFFUSION Outputs float array with diffusion coefficient values.
% Y=MY_VSSDIFFUSION(T,OMEGA,A,D,M,P,T_SAMPLE) generates a float array
% containing diffusion coefficient value for each value of T using the
% VSS model.
%
% -- T must be a float array of temperature values in K.
% -- OMEGA must be the VHS parameter.
% -- A must be the alpha VSS parameter.
% -- D must be the d VSS parameter.
% -- M must be the reduced mass in amu.
% -- P must be the pressure in bar.
% -- T_SAMPLE must be the reference temp in K.
%
% See also RUN_DSMCCOEF
%kb = 1.380649E-23; %J/K
kb = 8.617333262E-5; %eV/K
cref = 2*(2.5-omega)*kb*T_sample/m;
fprintf('cref is %f\n',cref)
y = 157377.3718*30000*(a+1)*(pi^0.5)*(kb*T).*((pi*2*kb*T/m).^omega)/(16*a*gamma(3.5-omega)*...
    P*pi*(d^2)*(cref)^(omega - 0.5));
end

```

Published with MATLAB® R2024a

coulomb

```

function y = coulombchar(z1)
%COULOMBCHAR Outputs character array with fitting model.

```

```
% Y=COULOMBCHAR(Z1) generates a char array containing
% the Coulomb potential equation with the user-specified Z1.
%
% -- Z1 must be the integer atomic number of species 1.
%
% See also RUN_FITPOTENTIAL
eps_naught = 0.005526349406; %e^2 * eV^-1 * Angstrom^-1
Ke = 1/(4*pi*eps_naught);
y = sprintf("(%d)*%i*z2./x",Ke,z1);
end
```

Published with MATLAB® R2024a

coulombchar

```
function y = coulombchar(z1)
%COULOMBCHAR    Outputs character array with fitting model.
% Y=COULOMBCHAR(Z1) generates a char array containing
% the Coulomb potential equation with the user-specified Z1.
%
% -- Z1 must be the integer atomic number of species 1.
%
% See also RUN_FITPOTENTIAL
eps_naught = 0.005526349406; %e^2 * eV^-1 * Angstrom^-1
Ke = 1/(4*pi*eps_naught);
y = sprintf("(%d)*%i*z2./x",Ke,z1);
end
```

Published with MATLAB® R2024a

diffusion_coef

```
function y = diffusion_coef(well_depth,T,m1,m2,d,inttype,data)
%DIFFUSION_COEF    Outputs self-diffusion coefficient.
% Y=DIFFUSION_COEF(WELL_DEPTH,T,P,M1,M2,D,INTTYPE,DATA) generates a float
% array containing a self-diffusion coefficient value for each temperature
% using a Lennard-Jones potential (from "Khrapak, S.A. Accurate transport
% cross sections for the Lennard-Jones potential. Eur. Phys. J. D 68, 276
% (2014)"). Units of self-diffusion coefficient are cm^2 /s.
%
% -- WELL-DEPTH must be the LJ well-depth in Kelvin.
% -- T must be a float array containing temperature values in units of K.
% -- P must be the pressure in bar.
% -- M1 must be the mass of species 1 in amu.
% -- M2 must be the mass of species 2 in amu.
% -- D must be the sigma LJ parameter in units of Angstrom.
% -- INTTYPE must be a character string specifying if the integral will
% be exact or numerical.
% -- DATA must be a character string filepath that gives the location of
% the cross section data.
%
% See also RUN_TRANSPORTCS
Tjoul = T.*(1.380649e-23);
mu = (m1*m2)/(m1+m2);
```

```

mukg = mu*(1.66054e-27);
dm = d*(1e-10);
Tstar = T/well_depth;
if strcmp(inttype,'Exact LJ')
    y = (3*sqrt(2*pi)/16)*((Tjoul).^(3/2))./(((mukg)^(1/2))* ...
        ((dm)^2)*reduceddifint(Tstar)); %PD in units of Pa * m^2 / s
elseif strcmp(inttype,'Trapezoidal LJ')
    y = (3*sqrt(2*pi)/16)*((Tjoul).^(3/2))./(((mukg)^(1/2))* ...
        ((dm)^2)*reduceddifquad(Tstar,data)); %PD in units of Pa * m^2 / s
else
    disp('Invalid Integration Type. Please check input file and try again.')
end
end

function y = reduceddifint(Tstar)
%REDUCEDDIFINT    Outputs the reduced diffusion integral.
% Y=REDUCEDDIFINT(TSTAR) generates a float array containing a value
% for the exact reduced diffusion integral in Khrapak, S.A. Accurate transport
% cross sections for the Lennard-Jones potential (2014).
%
% -- TSTAR is float array containing the reduced temperature values.
y = zeros(size(Tstar));
ct = 1;
for t = Tstar
    reddifintegrand = @(x) 0.5*(x.^2).*exp(-x).*diffusioncs(1./(2*t*x));
    y(ct) = integral(reddifintegrand,0,Inf);
    ct = ct + 1;
end
end

function y = reduceddifquad(Tstar,data)
%REDUCEDDIFQUAD    Outputs the reduced diffusion integral using quadrature.
% Y=REDUCEDDIFQUAD(TSTAR,DATA) generates a float array containing a value
% for the reduced diffusion integral in Khrapak, S.A. Accurate transport
% cross sections for the Lennard-Jones potential (2014) using trapezoidal
% integration.
%
% -- TSTAR is float array containing the reduced temperature values.
% -- DATA is a char array containing the filepath of the cross-section
% data.
y = zeros(size(Tstar));
ct = 1;
csdata = readmatrix(data);
for t = Tstar
    X = 1./(2*t*csdata(:,1));
    reddifintegrand = -0.25*(X.^2).*exp(-X).*csdata(:,2)./(t*csdata(:,1).^2);
    y(ct) = trapz(csdata(:,1),reddifintegrand);
    ct = ct + 1;
end
end

```

Published with MATLAB® R2024a

diffusioncs

```

function y = diffusioncs(beta)
%DIFFUSIONCS    Outputs float array with diffusion cross-section values.
% Y=DIFFUSIONCS(BETA) generates a float array containing diffusion

```

```

% cross-section value for each value of the scattering parameter beta
% using a Lennard_Jones potential.
%
% -- BETA must be a float array containing values for the dimensionless
% scattering parameter.
%
% See also VISCOSITYCS RUN_TRANSPORTCS
fLEd = 1 - 0.019.*(beta.^(-1)) + 0.038.*(beta.^(-2)) - 0.049.*(beta.^(-3)) ...
    + 0.015.*(beta.^(-4));
fHEd = 1 - 0.692.*(beta) + 9.594.*(beta.^(2)) - 8.284.*(beta.^(3)) ...
    - 2.355.*(beta.^(4));

y = zeros(size(beta));
%%Case 1: beta < 0.506
idx1 = beta < 0.506;
y(idx1) = 4.507.*(beta(idx1).^(1/6)).*fHEd(idx1);

%%Case 2: beta > 0.506
idx2 = beta > 0.506;
y(idx2) = 9.866.*(beta(idx2).^(1/3)).*fLEd(idx2);
end

```

Published with MATLAB® R2024a

difscatter

```

function y = difscatter(z1,z2,theta,E)
%DIFSCATTER    Outputs differential scattering cross section.
% Y=DIFSCATTER(Z1,Z2,THETA,E) generates a float array containing
% a differential scattering cross section value for each scattering
% angle using the Rutherford scattering model. Units of differential
% scattering cross section are Angstrom^2 / sr.
%
% -- Z1 must be the integer atomic number of species 1.
% -- Z2 must be the integer atomic number of species 2.
% -- THETA must be the float array containing scattering angle values in
% units of radians.
% -- E must be the incident energy as numeric char in units of eV.
%
% See also RUN_SCATTERINGINTEGRALS
eps_naught = 0.005526349406; %e^2 * eV^-1 * Angstrom^-1
Ke = 4*pi*eps_naught;
y = (z1*z2./(4*Ke*E.*sin(theta/2).^2)).^2;
end

```

Published with MATLAB® R2024a

distclose

```

function y = distclose(z1,z2,b,E)
%DISTCLOSE    Outputs distance of closest approach.
% Y=DISTCLOSE(Z1,Z2,B,E) generates a float array containing an
% impact parameter value for each scattering angle using the
% Rutherford scattering model. Units of distance of closest approach

```



```
% are Angstroms.
%
% -- Z1 must be the integer atomic number of species 1.
% -- Z2 must be the integer atomic number of species 2.
% -- B must be the float array containing impact parameter values in
% units of Angstroms.
% -- E must be the incident energy as numeric char in units of eV.
%
% See also RUN_SCATTERINGINTEGRALS
eps_naught = 0.005526349406; %e^2 * eV^-1 * Angstrom^-1
Ke = 1/(4*pi*eps_naught);
gamma = (Ke*z1*z2/E);
y = 0.5*(gamma+sqrt(gamma^2 + 4*(b.^2)));
end
```

Published with MATLAB® R2024a

impact

```
function y = impact(z1,z2,theta,E)
%IMPACT Outputs impact parameter.
% Y=IMPACT(Z1,Z2,THETA,E) generates a float array containing an
% impact parameter value for each scattering angle using the
% Rutherford scattering model. Units of impact parameter are Angstrom.
%
% -- Z1 must be the integer atomic number of species 1.
% -- Z2 must be the integer atomic number of species 2.
% -- THETA must be the float array containing scattering angle values in
% units of radians.
% -- E must be the incident energy as numeric char in units of eV.
%
% See also RUN_SCATTERINGINTEGRALS
eps_naught = 0.005526349406; %e^2 * eV^-1 * Angstrom^-1
Ke = 4*pi*eps_naught;
y = z1*z2.*cot(theta/2)./(2*Ke*E);
end
```

Published with MATLAB® R2024a

lj_124

```
function y = lj_124(eps_well,sig,r)
%LJ_124 Outputs Lennard Jones potential data.
% Y=LJ_124(EPS_WELL,SIG,R) generates a float array containing
% a potential energy value for each r value using the 12-4 Lennard Jones
% potential. Units of potential energy are eV.
%
% -- EPS_WELL must be the float LJ well depth.
% -- SIGMA must be the float distance at which the potential is zero.
% -- R must be the numerical array containing r values in units of
% Angstrom.
%
% See also RUN_CALCPOENTIAL
```

```

y = (0.5)*(3^1.5)*eps_well*(((sig./r).^12)-((sig./r).^4));
end

```

Published with MATLAB® R2024a

lj_126

```

function y = lj_126(eps_well,sig,r)
%lj_126    Outputs Lennard Jones potential data.
% Y=LJ_126(EPS_WELL,SIG,R) generates a float array containing
% a potential energy value for each r value using the 12-6 Lennard Jones
% potential. Units of potential energy are eV.
%
% -- EPS_WELL must be the float LJ well depth.
% -- SIGMA must be the float distance at which the potential is zero.
% -- R must be the numerical array containing r values in units of
% Angstrom.
%
% See also RUN_CALCPOTENTIAL
y = 4*eps_well*(((sig./r).^12)-((sig./r).^6));
end

```

Published with MATLAB® R2024a

magicscatter

```

function th = magicscatter(Ec,b,V,rm,z1,z2)
%MAGICSCATTER    Outputs float value for the scattering angle from the Magic Formula.
% Y=MAGICSCATTER(EC,B,V,RMIN,Z1,Z2) generates a float value for the
% scattering angle using the Magic Formula.
%
% -- EC must be the the energy in eV.
% -- B must be the impact parameter in Angstrom.
% -- V must be the potential function handle.
% -- RMIN is the distance of closest approach in Angstrom.
% -- Z1 is the atomic number of species 1.
% -- Z2 is the atomic number of species 2.
%
% See also RUN_SCATTERINGINTEGRALS
m1 = 1.008; %this needs to be updated for each run!!
m2 = 1.008; %this needs to be updated for each run!!
a = (0.46850)/(z1^(0.23) + z2^(0.23));
epschar = (a*Ec)/(z1*z2*14.4);
Vprmin = zbllderivative(z1,z2,rm);
thArg = (Bvar(b,a) + Rcvar(rhovar(Ec,V(rm),Vprmin),a) + ...
    Deltavar(epschar,Bvar(b,a),R0var(rm,a)))/(R0var(rm,a) + Rcvar(rhovar(Ec,V(rm),Vprmin),a));
th = 2*acos(thArg);
disp(th)
end

function y = Bvar(b,a)
y = b/a;
end

```

```

function y = Rcvvar(rhovar,a)
    y = rhovar/a;
end

function y = rhovar(E,Vrmin,Vprmin)
    y = -2*(E-Vrmin)/Vprmin;
end

function y = R0var(rmin,a)
    y = rmin/a;
end

function y = Deltavar(epschar,Bvar,R)
    C1 = 0.9923;
    C2 = 0.01162;
    C3 = 0.007122;
    C4 = 9.307;
    C5 = 14.81;
    alpha = 1 + C1/sqrt(epschar);
    beta = (C2 + sqrt(epschar))/(C3 + sqrt(epschar));
    A = 2*alpha*epschar*(Bvar^beta);
    gammavar = (C4 + epschar)/(C5 + epschar);
    G = gammavar/(sqrt(1+A^2)-A);
    y = A*(R-Bvar)/(1+G);
end

```

Published with MATLAB® R2024a

morse

```

function y = morse(rm,eps_well,k,r)
%MORSE    Outputs MORSE potential data.
%
%   Y=MORSE(RM,EPS_WELL,K,R) generates a float array containing
%   a potential energy value for each r value using the Morse potential.
%   Units of potential energy are eV.
%
%   -- RM must be the float distance at which the potential is at a minimum.
%   -- EPS_WELL must be the float Morse well depth.
%   -- K must be the force constant at the well minimum in units of eV/Angstrom^2.
%   -- R must be the numerical array containing r values in units of
%   Angstrom.
%
%   See also RUN_CALCPOENTIAL
    a = sqrt(k/(2*eps_well));
    y = eps_well*(exp(-2*a.*(r-rm))-2*exp(-a.*(r-rm)));
end

```

Published with MATLAB® R2024a

numdiffusioncoef

```

function y = numdiffusioncoef(Tvals,m1,m2,csdatafile)
%NUMDIFFUSIONCOEF    Outputs float array with diffusion coefficient values.
%   Y=NUMDIFFUSIONCOEF(TVALS,M1,M2,P,CSDATAFILE) generates a float array

```

```

%   containing self-diffusion coefficient value for each value of Tvals by]
%   using trapezoidal integration.
%
%   -- TVALS must be an array of temperatures in Kelvin.
%   -- M1 must be mass in amu.
%   -- M2 must be mass in amu.
%   -- CSDATAFILE must be location of diffusion cross section data file.
%
%   See also NUMDIFFUSIONCS RUN_TRANSPORTCS
kb = 8.617333262E-5; %eV/K
mred = m1*m2/(m1+m2);
csdata = readmatrix(csdatafile);
Evals = csdata(:,1); %eV
csvals = csdata(:,2); %A^2
amu_to_kg = 1/(6.022e26);
ev_to_j = 1.60218e-19;
a_to_m = 1e-10;
atomic_to_si = (amu_to_kg^(-1/2))*(ev_to_j^(3/2))*(a_to_m^(-2));

y = atomic_to_si*(3/8)*((2*pi/mred)^(0.5))*((kb*Tvals).^(3/2)).*(1./(reddifquad(Tvals,Evals,csvals))); %PD in units of Pa * m^2
end

function y = reddifquad(Tvals,Evals,csvals)
kb = 8.617333262E-5; %eV/K
y = zeros(size(Tvals));
ct = 1;
for T = Tvals
    reddifintegrand = ((kb*T).^(-3)).*exp(-Evals/(kb*T)).*(Evals.^2).*csvals;
    y(ct) = trapz(Evals,reddifintegrand);
    fprintf('T = %f, omega = %f',T,y(ct)*0.5)
    ct = ct + 1;
end
end

```

Published with MATLAB® R2024a

numdiffusioncs

```

function y = numdiffusioncs(bvals,th)
%NUMDIFFUSIONCS   Outputs diffusion cross-section value.
%   Y=NUMDIFFUSIONCS(SCATTERDATAFILE) generates diffusion cross-section
%   value by integrating scattering angle vs impact param data for a
%   specific energy using TRAPZ.
%
%   -- SCATTERDATAFILE must be the filepath to the scattering angle vs
%   impact para data.
%
%   See also RUN_TRANSPORTCS
difcsintegrand = 2*pi*(1-cos(th)).*bvals;
y = trapz(bvals,difcsintegrand);
end

```

Published with MATLAB® R2024a

numstoppingcs

```

function y = numstoppingcs(E,m1,m2,diffusioncs)
%NUMSTOPPINGCS    Outputs stopping cross-section value.
%   Y=NUMSTOPPINGCS(SCATTERDATAFILE) generates a stopping cross-section
%   value using diffusion cs value, masses, and energy.
%
%   -- E must be energy in eV.
%   -- M1 must be the mass of species 1 in amu.
%   -- M2 must be the mass of species 2 in amu.
%   -- DIFFUSIONCS must be diffusion CS value for energy E.
%
%   See also RUN_TRANSPORTCS
yCM = 2*(m1*m2/(m1+m2)^2)*E*diffusioncs;
CMtoLab = (m1+m2)/m2;
y = yCM*CMtoLab;
end

```

Published with MATLAB® R2024a

numtotalcs

```

function y = numtotalcs(th_max,bvals,th)
%NUMTOTALCS    Outputs total cross-section value.
%   Y=NUMTOTALCS(SCATTERDATAFILE) generates a total cross-section value
%   for a specific energy by finding the intersection of TH_MAX and the
%   scattering angle vs impact parameter curve.
%
%   -- TH_MAX must be the angle in radians used to determine total CS.
%   -- SCATTERDATAFILE must be the filepath to the scattering angle vs
%   impact para data.
%
%   See also RUN_TRANSPORTCS

bfine = min(bvals):0.00001:max(bvals);
th_p = th - th_max;
vqth = interp1(bvals,th_p,bfine);
bmax = max(data_zeros(bfine,vqth));
y = pi*bmax^2;
end

function x0 = data_zeros(x, y)
% Identify indices where there is a sign change between consecutive elements
zero_crossings = find((y(1:end-1) .* y(2:end)) <= 0);

% Preallocate for the zero-crossing points
x0 = NaN(length(zero_crossings), 1);

% Loop through each detected zero-crossing for interpolation
for k = 1:length(zero_crossings)
    % Indices for interpolation points
    idx1 = zero_crossings(k);
    idx2 = idx1 + 1;

    % Linear interpolation between the points around zero-crossing
    b = [1, x(idx1); 1, x(idx2)] \ [y(idx1); y(idx2)];
    x0(k) = -b(1) / b(2); % Solving for x at y = 0
end

```

```
x0 = x0(~isnan(x0));  
end
```

Published with MATLAB® R2024a

numtotalcscheb

```
function y = numtotalcscheb(th_max,bvals,th)  
%NUMTOTALCSCHEB    Outputs total cross-section value.  
% Y=NUMTOTALCSCHEB(SCATTERDATAFILE) generates a total cross-section value  
% for a specific energy by finding the intersection of TH_MAX and the  
% scattering angle vs impact parameter curve using Chebfun rootfinding.  
%  
% -- TH_MAX must be the angle in radians used to determine total CS.  
% -- SCATTERDATAFILE must be the filepath to the scattering angle vs  
% impact para data.  
%  
% See also RUN_TRANSPORTCS  
th_p = th - th_max;  
p = spline(bvals,th_p);  
poly = @(r) ppval(p,r);  
chebtotals = chebfun(poly,[min(bvals),max(bvals)]);  
bmax = max(roots(chebtotals));  
y = pi*bmax^2;  
end
```

Published with MATLAB® R2024a

numvisccoeff

```
function y = numvisccoeff(Tvals,m1,m2,csdatafile)  
%NUMVISCCOEF    Outputs float array with viscosity coefficient values.  
% Y=NUMVISCCOEF(TVALS,M1,M2,CSDATAFILE) generates a float array  
% containing viscosity coefficient value for each value of Tvals by using  
% trapezoidal integration.  
%  
% -- TVALS must be an array of temperatures in Kelvin.  
% -- M1 must be mass in amu.  
% -- M2 must be mass in amu.  
% -- CSDATAFILE must be location of viscosity cross section data file.  
%  
% See also NUMDIFFUSIONCOEF RUN_TRANSPORTCS  
kb = 8.617333262E-5; %eV/K  
mred = m1*m2/(m1+m2);  
csdata = readmatrix(csdatafile);  
Evals = csdata(:,1); %eV  
csvals = csdata(:,2); %A^2  
% note that the 0.001629989 below is a unit conversion factor that converts  
% [amu^0.5 * Angstrom^-2 * eV^0.5] --> [kg^0.5 * m^-2 * Joules^0.5]  
y = (1E6)*0.001629989*(5/4)*((2*pi*mred)^(0.5))*((kb*Tvals).^(1/2)).*(1./(redviscquad(Tvals,Evals,csvals))); %microPa s  
end  
  
function y = redviscquad(Tvals,Evals,csvals)  
kb = 8.617333262E-5; %eV/K
```

```

y = zeros(size(Tvals));
ct = 1;
for T = Tvals
    redviscintegrand = ((kb*T).^(-4)).*exp(-Evals/(kb*T)).*(Evals.^3).*csvals;
    y(ct) = trapz(Evals,redviscintegrand);
    ct = ct + 1;
end
end

```

Published with MATLAB® R2024a

numvisccs

```

function y = numvisccs(bvals,th)
%NUMVISCCS    Outputs viscosity cross-section value.
% Y=NUMVISCCS(SCATTERDATAFILE) generates a viscosity cross-section
% value by integrating scattering angle vs impact param data for a
% specific energy using TRAPZ.
%
% -- SCATTERDATAFILE must be the filepath to the scattering angle vs
% impact para data.
%
% See also RUN_TRANSPORTCS
difcsintegrand = 2*pi*(1-cos(th).^2).*bvals;
y = trapz(bvals,difcsintegrand);
end

```

Published with MATLAB® R2024a

powerlaw

```

function y = powerlaw(a,k,r)
%POWERLAW    Outputs power law potential data.
% Y=POWERLAW(A,K,R) generates a float array containing a potential
% energy value for each r value using the basic power law potential.
% Units of potential energy are eV.
%
% -- A is the first power law coefficient.
% -- K is the exponent power law coefficient.
% -- R must be the numerical array containing r values in units of
% Angstrom.
%
% See also RUN_CALCPOENTIAL
y = a*r.^(-k);
end

```

Published with MATLAB® R2024a

run_calcpotential

```

function y = run_calcpotential(filepath,datafilepath)
%RUN_CALCPOTENTIAL   Reads input file and generates potential data.
%   RUN_CALCPOTENTIAL(FILEPATH,DATAFILEPATH) reads the user-specified input
%   file, generates potential data using the appropriate potential
%   function, and saves the data to a user-specified folder.
%
%   -- FILEPATH must specify the path to where input file is.
%   -- DATAFILEPATH must specify where to save the potential data.
%
%   See also COULOMB LJ_126 LJ_124 ZBL POWERLAW MORSE
run(filepath);
r = minR:Rstep:maxR;
if strcmp(Potential_Type,"Coulomb")
    data = coulomb(Z1,Z2,r);
    datapath = fullfile(datafilepath,'/coulombdata.csv');
    A = [r' data'];
    writematrix(A,datapath);
    y = datapath;
elseif strcmp(Potential_Type,"12-6 Lennard-Jones")
    data = lj_126(eps_well,sig,r);
    datapath = fullfile(datafilepath,'/126ljdata.csv');
    A = [r' data'];
    writematrix(A,datapath);
    y = datapath;
elseif strcmp(Potential_Type,"12-4 Lennard-Jones")
    data = lj_124(eps_well,sig,r);
    datapath = fullfile(datafilepath,'/124ljdata.csv');
    A = [r' data'];
    writematrix(A,datapath);
    y = datapath;
elseif strcmp(Potential_Type,"ZBL")
    data = zbl(Z1,Z2,r);
    datapath = fullfile(datafilepath,'/zbldata.csv');
    A = [r' data'];
    writematrix(A,datapath);
    y = datapath;
elseif strcmp(Potential_Type,"Morse")
    data = morse(rm,eps_well,k,r);
    datapath = fullfile(datafilepath,'/morsedata.csv');
    A = [r' data'];
    writematrix(A,datapath);
    y = datapath;
elseif strcmp(Potential_Type,"Power Law")
    data = powerlaw(a,k,r);
    datapath = fullfile(datafilepath,'/powerlawdata.csv');
    A = [r' data'];
    writematrix(A,datapath);
    y = datapath;
end
end

```

Published with MATLAB® R2024a

run_dsmcccoef

```

function y = run_dsmcccoef(inputfile,datapath)
%RUN_DSMCCOEF   Reads dsmc coef input file and calculates dsmc params.

```



```

% RUN_DSMCCOEFF(INPUTFILE,DATAPATH) reads the user-specified parameters
% from the input file and calculates DSMC params for the VSS and VHS
% models.
%
% -- INPUTFILE must specify the path to where input file is.
% -- DATAFILEPATH must specify where to save the param table.
%
% See also VHSCOEFF VHSVISCOSITY VSSCOEF VSSDIFFUSION
y = datapath;
run(inputfile);
mr = m1*m2/(m1+m2);
mrkg = mr/(6.022E26);
kb = 1.380649E-23; %J/K
cd(sprintf('%s',y))
mkdir dsmcfitdata
visccoefdata = readmatrix(viscdatafile);
visccoef = visccoefdata(:,2);
Tvals = visccoefdata(:,1);
Tfine = min(Tvals):0.01:max(Tvals);
vqvisc = interp1(Tvals,visccoef,Tfine);
delta = (max(Tvals) - min(Tvals))/N;
offset = min(Tvals);
dsmcdatapath = fullfile(datapath,'/dsmccoefftable.csv');
A = [];
C = [Tfine' vqvisc'];
interpviscdata = fullfile(datapath,'/dsmcfitdata/dsmcfitviscdata.csv');
writematrix(C,interpviscdata);
for i = 1:N
    minT = offset;
    maxT = minT + delta;
    excludeT = ((Tfine < minT) | (Tfine > maxT));
    fitT = Tfine;
    fitT(excludeT) = [];
    T_sample = 0.5*(minT + maxT);
    vhscoefs = my_VHScoeff(fitT,T_sample,excludeT,vqvisc,tol);
    mu_sample = vhscoefs(1);
    omega = vhscoefs(2);
    dcollision = (1E10)*sqrt(15*sqrt(mrkg*kb*T_sample/pi)/...
        (2*(5-2*omega)*(7-2*omega)*(mu_sample/(1E6))))); %Angstrom
    viscfits = my_VHSviscosity(fitT,omega,T_sample,mu_sample);
    viscfitsdatapath = fullfile(datapath,sprintf('/dsmcfitdata/viscfitsdata_n=%i.csv',i));
    B1 = [fitT' viscfits'];
    writematrix(B1,viscfitsdatapath);
    A = [A; minT maxT mu_sample T_sample omega dcollision];
    offset = offset + delta;
end
A = array2table(A,'VariableNames',{'Tmin','Tmax','mu_ref','T_ref','omega','collision_diam'});
disp(A)
writetable(A,dsmcdatapath);
end

```

Published with MATLAB® R2024a

run_fitpotential

```

function y = run_fitpotential(filepath,datapath)
%RUN_FITPOTENTIAL Reads fitting input file and generates cfit object.
% RUN_FITPOTENTIAL(FILEPATH,DATAFILEPATH) reads the user-specified

```

```

% fitting parameters from the fitting input file, creates a fitttype based
% off those parameters using the appropriate character function or
% string, and then creates a fit object using the FIT function.
%
% -- FILEPATH must specify the path to where input file is.
% -- DATAFILEPATH must specify where the potential data is saved.
%
% See also COULOMBCHAR ZBLCHAR
run(filepath);
if strcmp(Potential_Type,"Coulomb")
    ft = fitttype(coulombchar(Z1),dependent="y",independent="x",...
        coefficients="z2");
    t = readmatrix(datapath);
    excludex = ((t(:,1) < minR) | (t(:,1) > maxR));
    xvals = t(:,1);
    yvals = t(:,2);
    xvals(excludex) = [];
    yvals(excludex) = [];
    y = fit(xvals,yvals,ft,'Exclude',(xvals<minR)&(xvals>maxR),...
        'TolFun',tol,'Lower',minZ2,'Upper',maxZ2);
elseif strcmp(Potential_Type,"ZBL")
    ft = fitttype(zblchar(Z1),dependent="y",independent="x",...
        coefficients="z2");
    t = readmatrix(datapath);
    excludex = ((t(:,1) < minR) | (t(:,1) > maxR));
    xvals = t(:,1);
    yvals = t(:,2);
    xvals(excludex) = [];
    yvals(excludex) = [];
    y = fit(xvals,yvals,ft,'Exclude',(xvals<minR)&(xvals>maxR),...
        'TolFun',tol,'Lower',minZ2,'Upper',maxZ2);
elseif strcmp(Potential_Type,"12-6 Lennard-Jones")
    t = readmatrix(datapath);
    excludex = ((t(:,1) < minR) | (t(:,1) > maxR));
    xvals = t(:,1);
    yvals = t(:,2);
    xvals(excludex) = [];
    yvals(excludex) = [];
    ft = fitttype("4*eps_well*((sigma./x).^12)-((sigma./x).^6)",dependent="y",...
        independent="x",coefficients=["eps_well" "sigma"]);
    y = fit(xvals,yvals,ft,...
        'TolFun',tol,'Lower',[min_eps min_sigma],...
        'Upper',[max_eps max_sigma],'StartPoint',[eps_start sigma_start]);
elseif strcmp(Potential_Type,"12-4 Lennard-Jones")
    t = readmatrix(datapath);
    excludex = ((t(:,1) < minR) | (t(:,1) > maxR));
    xvals = t(:,1);
    yvals = t(:,2);
    xvals(excludex) = [];
    yvals(excludex) = [];
    ft = fitttype("0.5*(3^1.5)*eps_well*((sigma./x).^12)-((sigma./x).^4)",dependent="y",...
        independent="x",coefficients=["eps_well" "sigma"]);
    y = fit(xvals,yvals,ft,...
        'TolFun',tol,'Lower',[min_eps min_sigma],...
        'Upper',[max_eps max_sigma],'StartPoint',[eps_start sigma_start]);
elseif strcmp(Potential_Type,"Morse")
    t = readmatrix(datapath);
    excludex = ((t(:,1) < minR) | (t(:,1) > maxR));
    xvals = t(:,1);
    yvals = t(:,2);
    xvals(excludex) = [];
    yvals(excludex) = [];

```

```

ft = fittype("eps_well*(exp(-2*sqrt(k/(2*eps_well)).*(x-rm))-2*exp(-sqrt(k/(2*eps_well)).*(x-rm)))", ...
    dependent="y",independent="x",coefficients=["rm" "eps_well" "k"]);
y = fit(xvals,yvals,ft,...
    'TolFun',tol,'Lower',[min_rm min_eps min_k],...
    'Upper',[max_rm max_eps max_k],'StartPoint',[rm_start eps_start k_start]);
elseif strcmp(Potential_Type,"Power Law")
    t = readmatrix(datapath);
    excludex = ((t(:,1) < minR) | (t(:,1) > maxR));
    xvals = t(:,1);
    yvals = t(:,2);
    xvals(excludex) = [];
    yvals(excludex) = [];
    ft = fittype("a*x.^(-k)",dependent="y",...
        independent="x",coefficients=["a" "k"]);
    y = fit(xvals,yvals,ft,...
        'TolFun',tol,'Lower',[min_a min_k],...
        'Upper',[max_a max_k],'StartPoint',[a_start k_start]);
end
end

```

Published with MATLAB® R2024a

run_scatteringintegrals

```

function y = run_scatteringintegrals(filepath,datafilepath,progressBar)
%RUN_SCATTERINGINTEGRALS    Reads scattering input file and calculates scattering integrals.
% RUN_SCATTERINGINTEGRALS(FILEPATH,DATAFILEPATH) reads the user-specified
% scattering parameters from the scattering input file and calculates the
% differential scattering cross section, impact parameter, and distance
% of closest approach (for the Coulomb case), or calculates the
% scattering angle and distance of closest approach (general) case.
%
% -- FILEPATH must specify the path to where input file is.
% -- DATAFILEPATH must specify where to save the scattering integral data.
%
% See also IMPACT DIFSCATTER DISTCLOSE DOCAROOT GMQUADSCATTERINGANGLE
y = datafilepath;
run(filepath);
if logspace_on == 1
    minElog = log10(minE);
    maxElog = log10(maxE);
    Evals = logspace(minElog,maxElog,logstep);
elseif logspace_on == 0
    Evals = minE:Estep:maxE;
end
cd(sprintf('%s',y))

if strcmp(inttype,'Exact Coulomb')
    theta = theta_min:theta_step:theta_max;
    numESteps = numel(Evals);
    numthSteps = numel(theta);
    A = zeros(numthSteps,numESteps);
    B = zeros(numthSteps,numESteps);
    C = zeros(numthSteps,2*numESteps);
    colNames = strings(1,1+numESteps);
    A(:,1) = theta';
    B(:,1) = theta';
    colNames(1,1) = 'theta';

```

```

colNamesC = strings(1,2*numESteps);
for i = 1:numESteps
    E = Evals(i);
    if nargin > 2 && isvalid(progressBar)
        progressBar.Value = i / numESteps;
        progressBar.Message = sprintf('Running %f eV...', E);
    end
    mydifscatter = difscatter(Z1, Z2, theta, E);
    impactparam = impact(Z2, Z2, theta, E);
    doca = distclose(Z1, Z2, impactparam, E);
    colNames(i+1) = sprintf('E=%f',E);
    colNamesC(1,2*i-1) = sprintf('bval E=%f',E);
    colNamesC(1,2*i) = sprintf('doca E=%f',E);
    A(:,i+1) = mydifscatter';
    B(:,i+1) = impactparam';
    C(:,2*i-1) = impactparam';
    C(:,2*i) = doca';
end
ATable = array2table(A, 'VariableNames', colNames);
BTable = array2table(B, 'VariableNames', colNames);
CTable = array2table(C, 'VariableNames', colNamesC);
difscatterdatapath = fullfile(datafilepath, '/difscatterdata.csv');
impactparamdatapath = fullfile(datafilepath, '/impactparamdata.csv');
docadatapath = fullfile(datafilepath, '/docadata.csv');
writetable(ATable, difscatterdatapath);
writetable(BTable, impactparamdatapath);
writetable(CTable, docadatapath);

elseif strcmp(inttype, 'Numerical')
    run(fitfile);
    if strcmp(Potential_Type, 'Coulomb')
        potential = @(r) coulomb(Z1, z2_param, r);
    elseif strcmp(Potential_Type, '12-6 Lennard-Jones')
        potential = @(r) lj_126(eps_param, sigma_param, r);
    elseif strcmp(Potential_Type, '12-4 Lennard-Jones')
        potential = @(r) lj_124(eps_param, sigma_param, r);
    elseif strcmp(Potential_Type, 'ZBL')
        potential = @(r) zbl(Z1, z2_param, r);
    elseif strcmp(Potential_Type, 'Morse')
        potential = @(r) morse(rm_param, eps_param, k_param, r);
    elseif strcmp(Potential_Type, 'Power Law')
        potential = @(r) powerlaw(a_param, k_param, r);
    end
    if blogspace_on == 1
        minblog = log10(bmin);
        maxblog = log10(bmax);
        bvals = logspace(minblog, maxblog, blogstep);
    elseif blogspace_on == 0
        bvals = bmin:bstep:bmax;
    end

    numESteps = numel(Evals);
    numBSteps = numel(bvals);
    A = zeros(numBSteps, numESteps);
    B = zeros(numBSteps, numESteps);
    colNames = strings(1, 1+numESteps);
    A(:,1) = bvals';
    B(:,1) = bvals';
    colNames(1,1) = 'bvals';

    for i = 1:numESteps
        E = Evals(i);

```

```

if nargin > 2 && isempty(progressBar)
    progressBar.Value = i / numESteps;
    progressBar.Message = sprintf('Running %f eV...', E);
end
docas = zeros(1, length(bvals));
th = zeros(1, length(bvals));
thmagic = zeros(1, length(bvals));
for j = 1:length(bvals)
    docas(j) = DOCAroot(E, bvals(j), potential, minroot, maxroot, chebfunpath);
    th(j) = GMquadScatteringAngle(potential, E, bvals(j), docas(j), 20);
    if strcmp(Potential_Type, 'ZBL')
        thmagic(j) = magicscatter(E, bvals(j), potential, docas(j), Z1, z2_param);
    end
end
colNames(i+1) = sprintf('E=%f', E);
A(:, i+1) = th';
B(:, i+1) = docas';
C = [bvals' thmagic'];
%magicscatterpath = fullfile(datafilepath, sprintf('/magicscatterdata/scatterangledata_%f.csv', E));
%writematrix(C, magicscatterpath)
end
ATable = array2table(A, 'VariableNames', colNames);
BTable = array2table(B, 'VariableNames', colNames);
scatterangdatapath = fullfile(datafilepath, '/scatterangledata.csv');
docadatapath = fullfile(datafilepath, '/docadata.csv');
writetable(ATable, scatterangdatapath);
writetable(BTable, docadatapath);
%writematrix(C, magicscatterpath)
end
end

```

Published with MATLAB® R2024a

run_transportCS

```

function run_transportCS(filepath, datafilepath)
%RUN_TRANSPORTCS Reads cross-section input file and calculates transport cross-sections.
% RUN_TRANSPORTCS(FILEPATH, DATAFILEPATH) reads the user-specified
% cross-section (CS) parameters from the cross-section input file and
% calculates the diffusion CS and viscosity CS (for the exact LJ case in
% "Khrapak, S.A. Accurate transport cross sections for the Lennard-Jones
% potential. Eur. Phys. J. D 68, 276 (2014)") or the diffusion CS,
% viscosity CS, stopping CS, and total CS (for the general case).
%
% -- FILEPATH must specify the path to where input file is.
% -- DATAFILEPATH must specify where to save the CS data.
%
% See also DIFFUSIONCS VISCOSITYCS NUMDIFFUSIONCS NUMVISCCS
% NUMTOTALCS NUMSTOPPINGCS
run(filepath);
if logspace_on == 1
    minElog = log10(minE);
    maxElog = log10(maxE);
    Evals = logspace(minElog, maxElog, logstep);
elseif logspace_on == 0
    Evals = minE:Estep:maxE;
end
if strcmp(inttype, 'Exact LJ')

```

```

    beta = welldepth./(2*Evals);
    difcs = diffusioncs(beta);
    visccs = viscositycs(beta);
    A = [Evals' difcs'];
    B = [Evals' visccs'];

elseif strcmp(inttype,'Numerical')
    difcs = zeros(1,length(Evals));
    visccs = zeros(1,length(Evals));
    stoppingcs = zeros(1,length(Evals));
    totalcs = zeros(1,length(Evals));
    for j = 1:length(Evals)
        disp(Evals(j))
        file = datafile;
        scatterdata = readmatrix(file);
        th = scatterdata(:,j+1);
        bvals = scatterdata(:,1);
        if strcmp(thetacutoff,'Quantum')
            docadata = readmatrix(docafile);
            doca = docadata(:,j+1)*(1e-10); %m
            hbar = 1.054571817E-34; %J*s
            m_redamu = m1*m2/(m1+m2);
            m_red = m_redamu/(6.022E26); %kg
            E = Evals(j)*(1.60218E-19); %J
            v_cm = sqrt(2*E/m_red);
            lam_bar = hbar/(m_red*v_cm);
            th_c = lam_bar./doca;
        elseif strcmp(thetacutoff,'Manual')
            th_c = th_max;
        end
        difcs(j) = numdiffusioncs(bvals,th);
        visccs(j) = numvisccs(bvals,th);
        stoppingcs(j) = numstoppingcs(Evals(j),m1,m2,diffusioncs(j));
        totalcs(j) = numtotalcs(th_c,bvals,th);
    end
    CMtoLab = (m1+m2)/m2;
    A = [Evals' difcs'];
    B = [Evals' visccs'];
    C = [(CMtoLab*Evals)' stoppingcs'];
    D = [Evals' totalcs'];
    stoppingcsdatapath = fullfile(datafilepath,'/stoppingcsdata.csv');
    totalcsdatapath = fullfile(datafilepath,'/totalcsdata.csv');
    writematrix(C,stoppingcsdatapath);
    writematrix(D,totalcsdatapath);
end
diffusioncsdatapath = fullfile(datafilepath,'/diffusioncsdata.csv');
viscositycsdatapath = fullfile(datafilepath,'/viscositycsdata.csv');
writematrix(A, diffusioncsdatapath);
writematrix(B, viscositycsdatapath);

end

```

Published with MATLAB® R2024a

run_transportcoefs

```

function y = run_transportcoefs(filepath,datafilepath)
%RUN_TRANSPORTCOEFS    Reads transport input file and calculates transport coefs.

```

```

% RUN_TRANSPORTCOEFS(FILEPATH,DATAFILEPATH) reads the user-specified
% transport parameters from the transport input file and calculates the
% self-diffusion or viscosity coefficient.
%
% -- FILEPATH must specify the path to where input file is.
% -- DATAFILEPATH must specify where to save the coef data.
%
% See also DIFFUSIONCOEF VISCCOEF NUMDIFFUSIONCOEF NUMVISCCOEF
%disp(filepath)
run(filepath);
Tvals = minT:Tstep:maxT;
if strcmp(inttype,'Exact LJ') | strcmp(inttype,'Trapezoidal LJ')
    difcoef = diffusion_coef(welldepth,Tvals,m1,m2,d,inttype,diffusiondatafile);
    diffusioncoefdatapath = fullfile(datafilepath,'/diffusioncoefdata.csv');
    A = [Tvals' difcoef'];
    writematrix(A, diffusioncoefdatapath);
    viscositycoef = visc_coef(welldepth,Tvals,m1,m2,d,inttype,viscositydatafile);
    visccoefdatapath = fullfile(datafilepath,'/viscositycoefdata.csv');
    B = [Tvals' viscositycoef'];
    writematrix(B, visccoefdatapath);
    y = datafilepath;
elseif strcmp(inttype,'Numerical')
    diffusioncoef = numdiffusioncoef(Tvals,m1,m2,diffusiondatafile);
    diffusioncoefdatapath = fullfile(datafilepath,'/diffusioncoefdata.csv');
    A = [Tvals' diffusioncoef'];
    writematrix(A, diffusioncoefdatapath);
    visccoef = numvisccoef(Tvals,m1,m2,viscositydatafile);
    visccoefdatapath = fullfile(datafilepath,'/viscositycoefdata.csv');
    B = [Tvals' visccoef'];
    writematrix(B, visccoefdatapath);
    coefdatapath = datafilepath;
    y = coefdatapath;
end
end

```

Published with MATLAB® R2024a

run_vhscoef

```

function y = run_vhscoef(inputfile,datapath)
%RUN_VHSCOEFS Reads VHS coef input file and calculates VHS params.
% RUN_DSMCCOEFS(INPUTFILE,DATAPATH) reads the user-specified parameters
% from the input file and calculates DSMC params for the VHS
% model.
%
% -- INPUTFILE must specify the path to where input file is.
% -- DATAFILEPATH must specify where to save the param table.
%
% See also VHSCOEFS VHSVISCOSITY
y = datapath;
run(inputfile);
%mr = m1*m2/(m1+m2);
mr = m1;
mrkg = mr/(6.022E26);
kb = 1.380649E-23; %J/K
cd(sprintf('%s',y))
mkdir vhsfitdata
visccoefdata = readmatrix(viscdatafile);

```

```

visccoef = visccoefdata(:,2);
Tvals = visccoefdata(:,1);
Tfine = min(Tvals):0.01:max(Tvals);
vqvisc = interp1(Tvals,visccoef,Tfine);
delta = (max(Tvals) - min(Tvals))/N;
offset = min(Tvals);
vhsdatapath = fullfile(datapath,'/vhscoeftable.csv');
A = [];
C = [Tfine' vqvisc'];
interpviscdata = fullfile(datapath,'/vhsfitdata/vhsfitviscdata.csv');
writematrix(C,interpviscdata);
for i = 1:N
    minT = offset;
    maxT = minT + delta;
    excludeT = ((Tfine < minT) | (Tfine > maxT));
    fitT = Tfine;
    fitT(excludeT) = [];
    T_sample = 0.5*(minT + maxT);
    mu_sample = interp1(Tfine, vqvisc, T_sample);
    vhscoefs = VHScoef(fitT,T_sample,mu_sample,excludeT,vqvisc,tol);
    omega = vhscoefs(2);
    dcollision = (1E10)*sqrt(15*sqrt(mrkg*kb*T_sample/pi)/...
        (2*(5-2*omega)*(7-2*omega)*(mu_sample/(1E6)))); %Angstrom
    viscfits = VHSviscosity(fitT,omega,T_sample,mu_sample);
    viscfitsdatapath = fullfile(datapath,sprintf('/vhsfitdata/viscfitsdata_n=%i.csv',i));
    B1 = [fitT' viscfits'];
    writematrix(B1,viscfitsdatapath);
    A = [A; minT maxT mu_sample T_sample omega dcollision];
    offset = offset + delta;
end
A = array2table(A,'VariableNames',{'Tmin','Tmax','mu_ref','T_ref','omega','collision_diam'});
disp(A)
writetable(A,vhsdatapath);
end

```

Published with MATLAB® R2024a

run_vsscoef

```

function y = run_vsscoef(inputfile,datapath)
%RUN_VSSCOEF    Reads VSS coef input file and calculates VSS params.
%   RUN_DSMCCOEFF(INPUTFILE,DATAPATH) reads the user-specified parameters
%   from the input file and calculates DSMC params for the VSS
%   model.
%
%   -- INPUTFILE must specify the path to where input file is.
%   -- DATAFILEPATH must specify where to save the param table.
%
%   See also VSSALPHA VSSDIAMETER VSSCONVERGENCE
y = datapath;
run(inputfile);
%mr = m1*m2/(m1+m2);
mr = m1;
mrkg = mr/(6.022E26);
molarkg = mr/(1E3);
difcoefdata = readmatrix(diffusiondatafile);
vhscoefdata = readmatrix(vhstablefile);
viscvals = vhscoefdata(:,3)*(1E-6);

```



```

omegavals = vhscoefdata(:,5);
alphavals = alphaguess*ones(length(omegavals),1);
Tvals = vhscoefdata(:,4);
difcoef = difcoefdata(:,2);
difcoef_T = difcoefdata(:,1);
dif_sample = interp1(difcoef_T, difcoef, Tvals);
max_iter = 100;
iter = 0;
while VSSconvergence(alphavals,omegavals,molarkg,Tvals,dif_sample,viscvals,tol) && iter < max_iter
    diams = VSSdiameter(alphavals,omegavals,mrkg,Tvals,viscvals);
    newalpha = VSSalpha(omegavals,mrkg,molarkg,Tvals,dif_sample,diams);
    alphavals = newalpha;
    iter = iter + 1;
end
if iter == max_iter
    warning('Reached maximum number of iterations without convergence.');
```

```

end
diams = VSSdiameter(alphavals,omegavals,mrkg,Tvals,viscvals);
diams_angstrom = diams*(1E10);
A = [Tvals alphavals diams_angstrom];
vssdatapath = fullfile(datapath,'/VSScoefstable.csv');
A = array2table(A,'VariableNames',{'Reference Temp (K)', 'alpha','collision_diam'});
disp(A)
writetable(A,vssdatapath);
end

```

Published with MATLAB® R2024a

visc_coef

```

function y = visc_coef(well_depth,T,m1,m2,d,inttype,data)
%VISC_COEF    Outputs viscosity coefficient.
% Y=VISC_COEF(WELL_DEPTH,T,M1,M2,D,INTTYPE,DATA) generates a float
% array containing a viscosity coefficient value for each temperature
% using a Lennard-Jones potential (from "Khrapak, S.A. Accurate transport
% cross sections for the Lennard-Jones potential. Eur. Phys. J. D 68, 276
% (2014)"). Units of viscosity coefficient are micro-Pascal*s.
%
% -- WELL-DEPTH must be the LJ well-depth in Kelvin.
% -- T must be a float array containing temperature values in units of K.
% -- M1 must be the mass of species 1 in amu.
% -- M2 must be the mass of species 2 in amu.
% -- D must be the sigma LJ parameter in units of Angstrom.
% -- INTTYPE must be a character string specifying if the integral will
% be exact or numerical.
% -- DATA must be a character string filepath that gives the location of
% the cross section data.
%
% See also RUN_TRANSPORTCS
Tjoul = T.*(1.380649e-23);
mu = (m1*m2)/(m1+m2);
mukg = mu*(1.66054e-27);
dm = d*(1e-10);
Tstar = T/well_depth;
if strcmp(inttype,'Exact LJ')
y = (1e6)*(5*sqrt(2*pi)/8)*(((mukg)^(1/2))*(Tjoul).^(1/2))./(((dm)^2) ...
    *reducedviscint(Tstar));
elseif strcmp(inttype,'Trapezoidal LJ')

```

```

y = (1e6)*(5*sqrt(2*pi)/8)*(((mukg)^(1/2))*(Tjou1).^(1/2))./(((dm)^2) ...
*reducedvisquad(Tstar,data));
else
    disp('Invalid Integration Type. Please check input file and try again.')
end
end

function y = reducedviscint(Tstar)
%REDUCEDVISCINT    Outputs the reduced viscosity integral.
% Y=MY_REDUCEVISCINT(TSTAR) generates a float array containing a value
% for the exact reduced viscosity integral in Khrapak, S.A. Accurate
% transport cross sections for the Lennard-Jones potential (2014).
%
% -- TSTAR is float array containing the reduced temperature values.
y = zeros(size(Tstar));
ct = 1;
for t = Tstar
    redviscintegrand = @(x) 0.5*(x.^3).*exp(-x).*viscositycs(1./(2*t*x));
    y(ct) = integral(redviscintegrand,0,Inf);
    ct = ct + 1;
end
end

function y = reducedviscquad(Tstar,data)
%REDUCEDVISCQUAD    Outputs the reduced viscosity integral using quadrature.
% Y=MY_REDUCEVISCQUAD(TSTAR,DATA) generates a float array containing a value
% for the reduced viscosity integral in Khrapak, S.A. Accurate transport
% cross sections for the Lennard-Jones potential (2014) using trapezoidal
% integration.
%
% -- TSTAR is float array containing the reduced temperature values.
% -- DATA is a char array containing the filepath of the cross-section
% data.
y = zeros(size(Tstar));
ct = 1;
csdata = readmatrix(data);
for t = Tstar
    X = 1./(2*t*csdata(:,1));
    redviscintegrand = -0.25*(X.^3).*exp(-X).*csdata(:,2)./(t*csdata(:,1).^2);
    y(ct) = trapz(csdata(:,1),redviscintegrand);
    ct = ct + 1;
end
end

```

Published with MATLAB® R2024a

viscositycs

```

function y = viscositycs(beta)
%VISCOSITYCS    Outputs float array with viscosity cross-section values.
% Y=VISCOSITYCS(BETA) generates a float array containing viscosity
% cross-section value for each value of the scattering parameter beta
% using a Lennard-Jones potential.
%
% -- BETA must be a float array containing values for the dimensionless
% scattering parameter.
%
% See also DIFFUSIONCS RUN_TRANSPORTCS

```

```

fLEeta = 1;
fHEeta = 1 - 2.229.*(beta) + 35.967.*(beta.^(2)) - 86.490.*(beta.^(3)) ...
    + 60.335.*(beta.^(4));

y = zeros(size(beta));
%%Case 1: beta < 0.491
idx1 = beta < 0.491;
y(idx1) = 3.599.*(beta(idx1).^(1/6)).*fHEeta(idx1);

%%Case 2: beta > 0.491
idx2 = beta > 0.491;
y(idx2) = 7.480.*(beta(idx2).^(1/3)).*fLEeta;
end

```

Published with MATLAB® R2024a

zbl

```

function y = zbl(z1,z2,r)
%ZBL    Outputs ZBL potential data.
%   Y=ZBL(Z1,Z2,R) generates a float array containing
%   a potential energy value for each r value using the ZBL potential.
%   Units of potential energy are eV.
%
%   -- Z1 must be the integer atomic number of species 1.
%   -- Z2 must be the integer atomic number of species 2.
%   -- R must be the numerical array containing r values in units of
%   Angstrom.
%
%   See also RUN_CALCOTPOTENTIAL
eps_naught = 0.005526349406; %e^2 * eV^-1 * Angstrom^-1
Ke = 4*pi*eps_naught;
a = (0.46850)/(z1^(0.23) + z2^(0.23));
y = phi(r/a).*z1*z2./(Ke.*r);
end

function y = phi(x)
y = 0.18175*exp(-3.19980*x) + 0.50986*exp(-0.94229*x) + ...
    0.28022*exp(-0.40290*x) + 0.02817*exp(-0.20162*x);
end

```

Published with MATLAB® R2024a

zblchar

```

function y = zblchar(z1)
%ZBLCHAR    Outputs character array with fitting model.
%   Y=ZBLCHAR(Z1) generates a char array containing the ZBL potential
%   equation with the user-specified Z1.
%
%   -- Z1 must be the integer atomic number of species 1.
%
%   See also RUN_FITPOTENTIAL
eps_naught = 0.005526349406; %e^2 * eV^-1 * Angstrom^-1

```

```

Ke = 1/(4*pi*eps_naught);
a = sprintf('(0.46850)/((%i)^(0.23) + z2^(0.23))',z1);
phi1 = sprintf('0.18175*exp(-3.19980*x/(%s)) + 0.50986*exp(-0.94229*x/(%s))',a,a);
phi2 = sprintf('0.28022*exp(-0.40290*x/(%s)) + 0.02817*exp(-0.20162*x/(%s))',a,a);
phi = [phi1 '+' phi2];
y = sprintf("(%d)*(%s)*%i*z2./x",Ke,phi,z1);
end

```

Published with MATLAB® R2024a

zblderivative

```

function y = zblderivative(z1,z2,r)
%ZBLDERIVATIVE    Outputs the derivative of a ZBL potential.
%   Y=ZBLDERIVATIVE(Z1,Z2,R) generates a float array containing
%   the derivative of the potential energy for each r value using the ZBL potential.
%   Units of potential energy are eV.
%
%   -- Z1 must be the integer atomic number of species 1.
%   -- Z2 must be the integer atomic number of species 2.
%   -- R must be the numerical array containing r values in units of
%   Angstrom.
%
%   See also RUN_CALCPOTENTIAL
eps_naught = 0.005526349406; %e^2 * eV^-1 * Angstrom^-1
k = 1/(4*pi*eps_naught);
a = (0.46850)/(z1^(0.23) + z2^(0.23));
y = (-k*z1*z2./(r.^2))*(0.18175*exp(-3.19980*r/a) + 0.50986*exp(-0.94229*r/a) + ...
    0.28022*exp(-0.40290*r/a) + 0.02817*exp(-0.20162*r/a)) + ...
    (k*z1*z2./(r*a))*((0.18175*-3.19980)*exp(-3.19980*r/a) + (0.50986*-0.94229)*exp(-0.94229*r/a) + ...
    (0.28022*-0.40290)*exp(-0.40290*r/a) + (0.02817*-0.20162)*exp(-0.20162*r/a));
end

```

Published with MATLAB® R2024a